# Intro to Spark

GENERAL ASSEMBLY

# Agenda

- What is Spark?
- How Spark works
- DataBricks
- Functional programming aside
- PySpark programming!

 © 2020 General Assembly

# What is Spark?

Apache Spark is an open-source **distributed cluster-computing framework**.

This means that it's a framework for spreading out intense calculations over several computers. It is also has **resiliency**, which means that if one of the computers fails, Spark will simply ask a working computer to finish its work.




 © 2020 General Assembly

# Why do we need this?

RAM (8GB)

 © 2020 General Assembly

# Why do we need this?




 © 2020 General Assembly

# Why do we need this?




 © 2020 General Assembly

# Why do we need this?



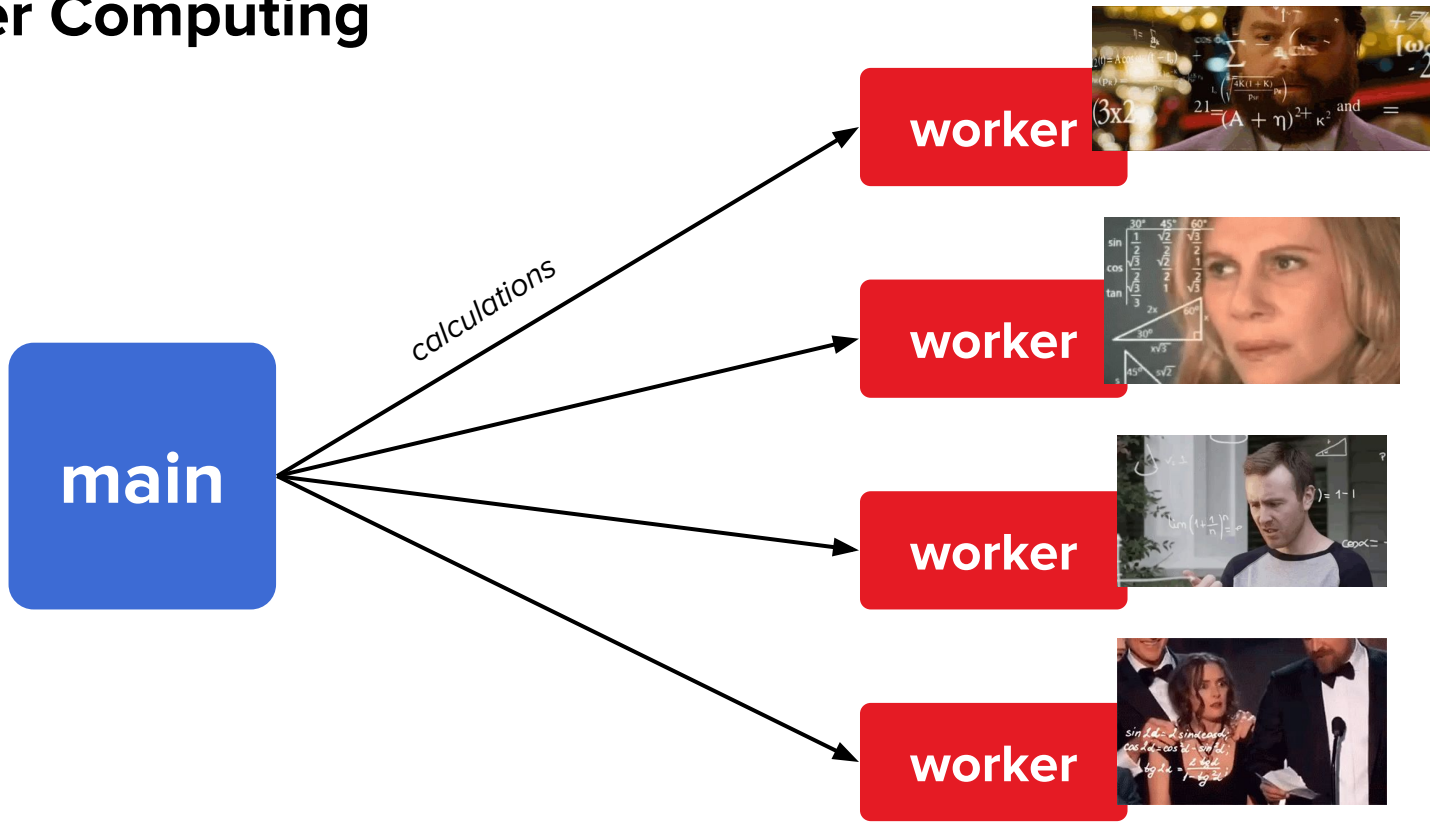

 © 2020 General Assembly

# Why do we need this?

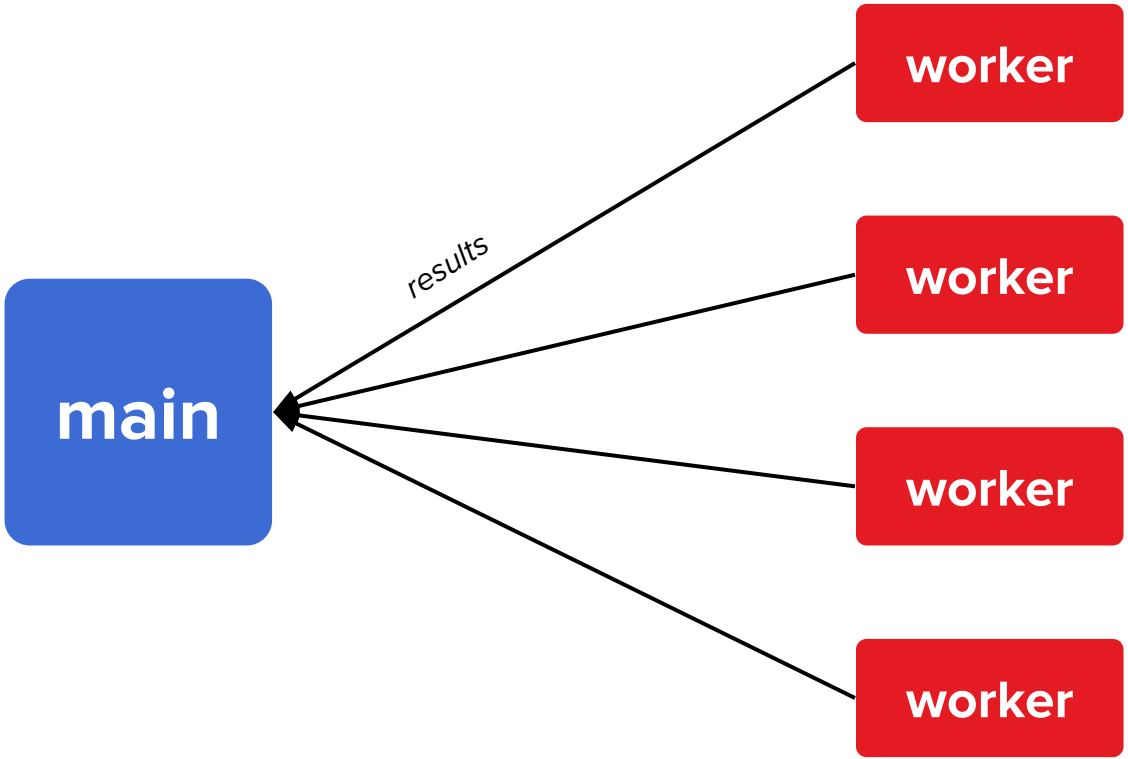





# Cluster Computing



# Cluster Computing




 © 2020 General Assembly

# How do we get Spark up and running?

# Languages

Spark is written in **Scala**, but it is implemented across several different languages. For the sake of only teaching one new thing at a time, we will use **PySpark**.











# DataBricks

In order to use Spark for its intended purpose, you'll need to spin up a **Spark server** and set it up to connect to your worker nodes.

This is difficult, time-consuming, and most often done by a DevOps person.

The specially-made AWS service for this is called **AWS Elastic MapReduce (EMR)**.

# DataBricks

The company **DataBricks** provides a user-friendly front-end for this whole process. We can pay them to set up EMR instances, and connect a notebook with PySpark pre-set up for us.

We'll use the free **Community Edition** for our lesson today!



databricks

# Functional Programming and Parallelization

# Tim, you're obsessed with FP, why are we talking about this again?!

It's important, I promise! Since PySpark is written in **Scala**, it follows some Scala programming patterns, even though we're accessing it through Python. Specifically you'll notice two things:

1. Scala variables, functions, and methods are written in **`camelCase()`**
2. Scala Spark uses **functional programming** relatively strictly

 © 2020 General Assembly

# Keep it short: What does FP mean for me?

**Fact 1:** Every Spark object is **immutable**. That is, there are no self-mutating "inplace" methods.

**Invalid Spark Pattern:**

```
df.someMethod(..., inplace=True)
```

**Valid Spark Pattern:**

```
df = df.someMethod(...)
```

# Keep it short: What does FP mean for me?

**Fact 2:** One of the big advantages of FP is that it's easily **parallelizable**. However, some DS tasks are not parallelizable. Keep an eye out for some actions that are either impossible or difficult to parallelize, such as:

- Sorting
- Computing the median (or any other percentile)
- Non-parallelizable ML models (neural nets are a big one!)

# Keep it short: What does FP mean for me?

**Fact 3:** Spark is **lazy**. No really, that's a technical term! It means that things don't get computed until the absolute last minute. Take this pseudo-example:

```
df = df.orderBy("col1")
df = df.filter(col("col2") > 0)
df = df.groupBy("col3").agg("col4")
print(df)
```

**When does the code actually run?**

# Let’s open up DataBricks and get started!



databricks